

Assignment I – FIO

Q0. (10%) Example

Results of q0.fio on **SSD**, output:

```
job1: (g=0): rw=read, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=psync,
iodepth=1
fio-3.36
Starting 1 process
```

```
job1: (groupid=0, jobs=1): err= 0: pid=36730: Tue Mar 17 13:31:55 2026
read: IOPS=14.9k, BW=58.3MiB/s (61.2MB/s)(186MiB/3189msec)
   clat (usec): min=38, max=592, avg=66.50, stdev=33.25
   lat (usec): min=38, max=592, avg=66.56, stdev=33.27
   clat percentiles (usec):
   |  1.00th=[  46],  5.00th=[  49], 10.00th=[  50], 20.00th=[  51],
   | 30.00th=[  52], 40.00th=[  54], 50.00th=[  56], 60.00th=[  60],
   | 70.00th=[  65], 80.00th=[  71], 90.00th=[  90], 95.00th=[ 120],
   | 99.00th=[ 225], 99.50th=[ 265], 99.90th=[ 367], 99.95th=[ 420],
   | 99.99th=[ 515]
   bw (  KiB/s): min=49128, max=69368, per=100.00%, avg=61044.00, stdev=8157.18, samples=6
   iops        : min=12282, max=17342, avg=15261.00, stdev=2039.30, samples=6
   lat (usec)  : 50=9.14%, 100=83.17%, 250=7.07%, 500=0.60%, 750=0.02%
   cpu         : usr=2.35%, sys=8.85%, ctx=47628, majf=0, minf=13
   IO depths   : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
   submit      : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
   complete    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
   issued rwts: total=47627,0,0,0 short=0,0,0,0 dropped=0,0,0,0
   latency     : target=0, window=0, percentile=100.00%, depth=1
```

Run status group 0 (all jobs):

```
  READ: bw=58.3MiB/s (61.2MB/s), 58.3MiB/s-58.3MiB/s (61.2MB/s-61.2MB/s), io=186MiB (195MB),
run=3189-3189msec
```

Disk stats (read/write):

```
 sdb: ios=45764/0, sectors=366112/0, merge=0/0, ticks=2494/0, in_queue=2494, util=72.63%
```

Q1. (10%) Read vs. Randread

Is there any significant difference between read/randread? Why or why not?
Please justify your answer in brief.

- Yes, there is a significant difference between the two workloads. The FIO benchmark data clearly demonstrates that sequential reads are substantially faster and more efficient than random reads on this storage device.
- The FIO output provides three primary performance metrics that justify this conclusion. In this test, the sequential workload outperformed the random workload by nearly a factor of two:

Metric	Sequential Read (read)	Random Read (randread)	Difference
IOPS	15.3k	8,497	~80% higher for sequential
Bandwidth	59.6 MiB/s	33.2 MiB/s	~80% higher for sequential
Avg Latency	65.15 μ s	117.07 μ s	~45% lower (faster) for sequential

- Sequential reads are faster because:
 - Read-Ahead Caching: Sequential reads are predictable, allowing the system to proactively fetch contiguous data blocks into high-speed cache, which maximizes throughput and minimizes latency.
 - FTL Mapping Overhead: Random reads are unpredictable, defeating caching and forcing the SSD controller to constantly look up physical locations in the Flash Translation Layer (FTL), adding computational delay.

Q2. (10%) Write vs. Randwrite

Is there any significant difference between write/randwrite? Why or why not?
Please justify your answer in brief.

- While there is a measurable difference between the two workloads, it is relatively minor (about 16%) and, somewhat counter-intuitively, the random writes actually performed slightly faster than the sequential writes in this specific test.

- The FIO output reveals that the randwrite job outperformed the sequential write job across all primary metrics:

Performance Metric	Sequential Write (write)	Random Write (randwrite)	Performance Gap
Throughput (Bandwidth)	35.2 MiB/s	41.1 MiB/s	~16% higher for random
IOPS	9,022 IOPS	10,538 IOPS	~16% higher for random
Average Latency	110.13 μ s	94.16 μ s	~14% lower (faster) for random

- Random write workload performed slightly faster than the sequential write workload because:
 - Log-Structured FTL: SSDs absorb all incoming writes into a high-speed buffer and physically write them sequentially to the flash memory, effectively eliminating the performance penalty for random logical writes
 - File Layout Overhead: The sequential write job ran first and suffered the initial system overhead of creating and allocating the 64MB test file, whereas the random write ran second and benefited from using the already-allocated space.

Q3. (10%) Forward write vs. Backward write

Is there any significant difference between forward/backward write? Why or why not? Please justify your answer in brief.

- The backward write actually performed significantly better in this test, achieving ~101k IOPS (393 MiB/s) compared to the forward write's ~64k IOPS (250 MiB/s).

Performance Metric	Forward Write	Backward Write	Performance Gap
Throughput (Bandwidth)	250 MiB/s	393 MiB/s	~57% higher for backward

IOPS	64,000 IOPS	101,000 IOPS	~58% higher for backward
Average Latency	237.00 μ s	156.48 μ s	~34% lower (faster) for backward

- This is due to the same FIO testing artifacts and SSD behaviors seen in your previous experiment:
 - **File Layout Penalty:** The forward_write job ran first and was forced to absorb the heavy file system overhead of creating and allocating the 64MB test file (indicated by the Laying out IO file log). The backward_write ran second on the already-allocated file, completely bypassing this system delay.
 - **SSD FTL Masking:** On a hard drive, backward writes would cause severe mechanical seek delays. However, on an SSD, the Flash Translation Layer (FTL) absorbs these backward logical block addresses into a high-speed write buffer and flushes them sequentially to the physical flash memory. This neutralizes any logical order penalty, allowing the lack of file-creation overhead to dominate the backward_write result.

Q4. (15%) Buffered read vs. Nonbuffered read

1. (10%) Is there any significant difference between buffered/nonbuffered sequential read? Why or why not? Please justify your answer in brief.

- Yes, there is a massive and significant difference. The sequential buffered read (direct=0) performed extraordinarily faster than the non-buffered read (direct=1).

Metric	Sequential Buffered Read (direct=0)	Sequential Non-buffered Read (direct=1)
Throughput	1,422 MiB/s	252 MiB/s
IOPS	364,000 IOPS	64,500 IOPS
Avg Latency	~42 μ s	~242 μ s

- **The Page Cache Advantage:** By setting direct=0, FIO allows the Linux operating system to utilize the Page Cache (RAM). Because the workload is perfectly sequential, the OS easily predicts the read pattern and aggressively pre-fetches the entire 64MB file from the physical disk into the system's memory.

- The throughput of 1,422 MiB/s and ultra-low latency of $\sim 42\mu\text{s}$ indicate that the benchmark is not actually reading from the SSD; it is reading directly from the vastly faster system RAM. In contrast, `direct=1` bypasses the OS cache entirely, forcing every I/O request to go directly to the physical storage device, revealing the true hardware limit of ~ 252 MiB/s.

2. (5%) Replace sequential read with random read. Is there any significant difference between buffered/nonbuffered random read? Why or why not? Please justify your answer in brief.

- Yes, there is a significant difference, but in the exact opposite direction: the random non-buffered read (`direct=1`) was dramatically faster than the random buffered read (`direct=0`).

Metric	Random Buffered Read (<code>direct=0</code>)	Random Non-buffered Read (<code>direct=1</code>)
Throughput	33.0 MiB/s	388 MiB/s
IOPS	8,445 IOPS	99,300 IOPS

- Random reads are inherently unpredictable, rendering the OS's read-ahead caching algorithms completely ineffective. Worse yet, when using `direct=0` with random reads, every request forces the OS to manage the Page Cache (checking if data is there, failing, fetching from disk, writing to cache, and returning to the application). This constant software overhead creates a severe bottleneck, tanking performance to 33 MiB/s.
- By using `direct=1`, the non-buffered workload bypasses the useless OS caching layer altogether. It sends the random I/O requests straight to the SSD controller, which is highly optimized to handle concurrent random lookups at an iodepth of 16, resulting in a much higher ~ 388 MiB/s.

Q5. (10%) LBA

Is there any bandwidth trend between these jobs? Why or why not? Please justify your answer in brief. You can also experiment with the size of the file.

- There is no spatial trend based on the file offsets, but there is a slight temporal trend (degradation over time) due to the execution order.

Job Name	File Offset	Execution Order	Throughput (Bandwidth)
job1	80%	1st	2,169 MiB/s
job2	60%	2nd	2,246 MiB/s
job3	40%	3rd	2,165 MiB/s
job4	20%	4th	2,103 MiB/s
job5	0%	5th	2,032 MiB/s

Justification:

- At speeds over 2,000 MiB/s, the storage device is clearly an NVMe SSD. If this were a mechanical Hard Disk Drive (HDD), you would see a massive physical trend where the 0% offset (the outer edge of the platter) performs much faster than the 80/100% offset (the inner tracks) due to angular velocity. Because SSDs use flash memory with internal FTL mapping, logical offsets have absolutely zero impact on physical retrieval speeds.
- Presence of Temporal Trend: The data shows a gradual decline in bandwidth as the test progresses from the 2nd job to the 5th job. Because these 1GB jobs are running back-to-back at extremely high speeds, this slight degradation is almost certainly caused by thermal throttling or controller saturation. As the NVMe drive heats up under sustained load, it slightly throttles performance to protect itself.

Q6. (15%) Blocksize

Question :

1. (10%) Is there any significant difference between 4k/1k read? Why or why not? Please justify your answer in brief.

- Yes, there is a very significant difference. The 4KB block size performed dramatically better than the 1KB block size, especially in terms of throughput.

Performance Metric	read_4k (bs=4k)	read_1k (bs=1k)	Performance Gap
Throughput (Bandwidth)	57.0 MiB/s	11.7 MiB/s	~387% higher for 4k
Operations Per Second (IOPS)	14,600 IOPS	11,900 IOPS	~22% higher for 4k

Average Latency	68.10 μ s	83.34 μ s	~18% lower (faster) for 4k
------------------------	---------------	---------------	-----------------------------------

Justification:

- **I/O Overhead Multiplier:** To read the total 64MB test file, the 4K test only had to issue 16,384 individual I/O requests. The 1K test had to issue exactly four times as many (65,536 requests). Each request carries system call and hardware protocol overhead. The 1K test spends most of its time processing this overhead rather than moving actual data.
- **Native Sector Alignment:** Modern storage devices (and operating systems) natively process data in minimum block sizes of 4KB (or larger internal pages). When the 1K test asks for 1KB, the hardware still has to fetch the full 4KB physical block and discard the rest. This wasted effort effectively tanks the bandwidth.

2. (5%) If you want to achieve the best performance in the condition above, how would you modify blocksize? Explain it briefly.

- To achieve the maximum possible performance (highest throughput), we can increase the block size significantly (e.g., bs=128k, bs=1m, or even bs=4m).
- Increasing the block size maximizes the "payload-to-overhead" ratio. By sending massive chunks of data with a single I/O request, we drastically reduce the number of system calls, context switches, and hardware interrupts.
- This allows the system to spend its resources purely on moving data, successfully saturating the maximum bandwidth capabilities of the underlying NVMe SSD.

Q7. (20%) Fastest nonbuffered read

Question : Please explain how you achieve the fastest 1G nonbuffered read in brief.

- The peak non-buffered read performance of 6,360 MiB/s (6.6 GB/s) was achieved by maximizing the parallel processing capabilities of the underlying NVMe SSD. This was accomplished through the combination of three specific FIO parameters:
- **Increased Block Size (bs=1m):** The block size was increased from a standard 4KB to 1MB. By doing so, the total number of I/O requests required to read the 1GB test

file was drastically reduced (dropping from over 262,000 requests to just 1,024). Consequently, CPU interrupts, context switching, and protocol overhead were minimized, allowing the storage bus to dedicate its bandwidth entirely to moving heavy data payloads.

- **Asynchronous I/O Engine (ioengine=libaio):** Instead of utilizing a synchronous engine that waits for one read to complete before requesting the next, Linux AIO was employed. This allowed multiple read requests to be submitted to the operating system simultaneously without blocking the application.
- **Deepened I/O Queue (iodepth=16):** To leverage the multiple flash memory chips and internal channels present in modern NVMe SSDs, an active queue of 16 concurrent 1MB requests was maintained at all times. By sustaining this deep queue, the SSD controller was successfully saturated, forcing all internal NAND channels to pull data concurrently to reach the hardware's maximum throughput limit.

Bonus (10%)

1. (5%) Compare with multiple kinds of storage devices

Is there any significant difference between read/randread on storage devices?

Why or why not? Please justify your answer in brief.

- **Yes**, there is a significant difference between sequential read (read) and random read (randread) operations, and the magnitude of this difference depends entirely on the underlying physical storage medium.
- **Hard Disk Drives (HDDs):** The performance gap between sequential and random operations is massive. HDDs rely on mechanical components (spinning platters and moving read/write actuator arms). Random reads require the mechanical arm to physically reposition for every operation, incurring severe "seek time" and "rotational latency." This drops random performance to just a few Megabytes per second, whereas sequential reads allow continuous data retrieval without moving the head.
- **Solid State Drives (SSDs):** The performance gap is significantly smaller but still present. Because SSDs use flash memory with no moving parts, the mechanical seek penalty is eliminated. However, random reads are still slower than sequential reads due to the Flash Translation Layer (FTL) overhead required to look up scattered logical-to-physical address mappings, as well as the SSD controller's inability to effectively use read-ahead prefetching for unpredictable data locations.

2. (5%) Find the specs of your own hardware that you tested on
Is your hardware running to spec? If not, could you come up with a possible theory?

Hardware Model: WD PC SN7100S SDFPMSL-1T00-1101

Disk Test Suite Average Results for WD PC SN7100S SDFPMSL-1T00-1101	
Sequential Read	6,508 MBytes/Sec
Sequential Write	5,353 MBytes/Sec
Random Seek Read Write (IOPS 32KQD20)	3,727 MBytes/Sec
IOPS 4KQD1	92 MBytes/Sec

Source: <https://www.harddrivebenchmark.net/hdd.php?hdd=WD+PC+SN7100S+SDFPMSL-1T00-1101&id=42963>

The hardware is running to specification for sequential workloads but underperforming in low-queue-depth random workloads.

For sequential reads, the observed performance of 6,669 MB/s actually slightly exceeds the reference benchmark of 6,508 MB/s, indicating the drive is fully capable of saturating its PCIe Gen4 interface under optimal conditions. However, the 4K QD1 read performance fell short, achieving only 59.8 MB/s compared to the referenced 92 MB/s.

The shortfall in 4K QD1 performance can be attributed to several factors:

- **File System vs. Raw Block Device Overhead:** Standard benchmark utilities often bypass the file system entirely by testing the raw block device. The FIO tests were executed on a pre-allocated file (filename=./fio_test_file) within a mounted file system, introducing software stack overhead that heavily impacts low-latency 4K operations.
- **CPU Power Management (C-States):** 4K QD1 performance is extremely sensitive to CPU single-thread performance and latency. If motherboard power-saving features or CPU sleep states (C-states) are enabled, the microsecond delays in waking the CPU cores can significantly reduce the maximum achievable 4K IOPS.
- **Thermal Throttling / Background Operations:** The FIO test suite contained consecutive heavy tests. It is highly probable that the SSD was managing elevated temperatures or performing background flash maintenance (garbage collection) during the 4K run, which penalizes random read latency.